CLOUD-03: AWS EKS Monitoring

> **Series:** CLOUD — Cloud Provider Integrations | **Notebook:** 3 of 8 | **Created:** March 2026 | **Last Updated:** 04/04/2026

Overview

This notebook provides a deep dive into monitoring Amazon Elastic Kubernetes Service (EKS) with Dynatrace. You will learn about EKS-specific monitoring considerations including node groups, Fargate profiles, CloudWatch Container Insights vs Dynatrace, DynaKube operator deployment on EKS, and namespace-level cost allocation patterns.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled

Requirement	Details
Permissions	`metrics.read`, `entities.read`, `logs.read`
AWS EKS Cluster	At least one EKS cluster with Dynatrace Operator installed
DynaKube Operator	v1.3+ deployed via Helm or kubectl
Prior Knowledge	CLOUD-01 fundamentals, basic Kubernetes concepts

1. EKS Monitoring Architecture

Monitoring EKS with Dynatrace involves multiple data paths:

Data Path	Source	What It Monitors
DynaKube Operator	OneAgent on each node	Pods, containers, processes, traces
AWS Cloud Integration	CloudWatch API via ActiveGate	EKS control plane, node group metrics
Kubernetes API	Dynatrace ActiveGate	Cluster events, deployments, workload state
Prometheus Integration	kube-state-metrics, node-exporter	Custom and detailed K8s metrics

EKS-Specific Considerations

- **Control plane is managed** — AWS manages the API server, etcd, and scheduler. You cannot install agents on control plane nodes.
- **Node groups vs Fargate** — Different monitoring approaches for each compute type.
- **VPC CNI** — EKS uses the Amazon VPC CNI plugin, which assigns VPC IP addresses to pods. This affects network monitoring.
- **IRSA (IAM Roles for Service Accounts)** — Recommended for granting DynaKube pods access to AWS resources.

2. DynaKube on EKS

The Dynatrace Operator (DynaKube) is the recommended way to monitor EKS workloads.

Deployment Modes

Mode	Description	Best For
CloudNativeFullStack	Full agent injection + infrastructure monitoring	Production clusters
ApplicationMonitoring	Code-level injection only (no infrastructure)	Shared/multi-tenant clusters
ClassicFullStack	DaemonSet-based (legacy)	Migration from classic deployment

DynaKube Configuration for EKS

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://&lt;environment-id&gt;.live.dynatrace.com/api
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          key: node-role
          operator: Exists
      nodeSelector:
        kubernetes.io/os: linux
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
```

Key EKS Deployment Notes

- Use **tolerations** to ensure OneAgent runs on all node groups, including tainted nodes
- Enable **kubernetes-monitoring** capability on ActiveGate for cluster-level metrics
- Use **nodeSelector** to exclude Windows nodes if running mixed clusters
- Consider **resource limits** to prevent OneAgent from consuming excessive node resources

3. Node Group Monitoring

EKS node groups are collections of EC2 instances that serve as Kubernetes worker nodes.

Node Group Types

Type	Monitoring Approach	Notes
Managed node groups	OneAgent DaemonSet + CloudWatch	Automatic scaling, patching
Self-managed nodes	OneAgent DaemonSet + CloudWatch	Full control, manual management
Karpenter nodes	OneAgent DaemonSet	Dynamic provisioning, varied instance types

Querying EKS Nodes

```
// List Kubernetes cluster entities
fetch dt.entity.kubernetes_cluster
| fieldsKeep id, entity.name, tags
| sort entity.name asc

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes K8S_CLUSTER
// | fieldsKeep id, name, tags
// | sort name asc
```

Node CPU and Memory Usage

```
// Node-level CPU usage over the last hour
timeseries nodeCpu = avg(dt.kubernetes.node.cpu_usage), from:-1h, by:
{dt.entity.kubernetes_node}
| fieldsAdd avgCpu = arrayAvg(nodeCpu)
| sort avgCpu desc
| limit 15
```

```
// Node memory working set over the last hour
timeseries nodeMemory = avg(dt.kubernetes.node.memory_working_set), from:-1h,
by:{dt.entity.kubernetes_node}
| fieldsAdd avgMemory = arrayAvg(nodeMemory)
| sort avgMemory desc
| limit 15
```

4. Fargate Profile Monitoring

AWS Fargate runs pods without managing nodes. This changes the monitoring approach significantly.

Fargate Monitoring Limitations

Capability	EC2 Nodes	Fargate
OneAgent DaemonSet	Supported	Not supported (no node access)
Code injection	Via DaemonSet	Via init container (ApplicationMonitoring mode)
Host metrics	Full (CPU, memory, disk, network)	Container metrics only
Process monitoring	Full	Limited
Network monitoring	Full	Limited

Monitoring Fargate Workloads

For Fargate pods, use **ApplicationMonitoring** mode in DynaKube:

```
spec:
  oneAgent:
    applicationMonitoring:
      useCSIDriver: false  # Fargate doesn't support CSI
```

> **Note:** Fargate pods cannot use the CSI driver. Set `useCSIDriver: false` and Dynatrace will inject via init containers instead.

5. EKS Metrics with DQL

Pod CPU and Memory by Namespace

```
// Pod CPU usage by namespace over the last hour
timeseries podCpu = avg(dt.kubernetes.container.cpu_usage), from:-1h, by:
{k8s.namespace.name}
| fieldsAdd avgCpu = arrayAvg(podCpu)
| sort avgCpu desc
| limit 10
```

```
// Container memory working set by namespace over the last hour
timeseries containerMem = avg(dt.kubernetes.container.memory_working_set),
from:-1h, by:{k8s.namespace.name}
| fieldsAdd avgMem = arrayAvg(containerMem)
| sort avgMem desc
| limit 10
```

Pod Status Overview

```
// Kubernetes events related to pod issues in the last 6 hours
fetch events, from:-6h
| filter event.kind == "K8S_EVENT"
| filter event.type == "Warning"
| summarize event_count = count(), by:{event.reason, k8s.namespace.name}
| sort event_count desc
| limit 20
```

6. Container Insights vs Dynatrace

Many EKS users start with CloudWatch Container Insights. Here is how it compares to Dynatrace:

Capability	CloudWatch Container Insights	Dynatrace
Setup complexity	AWS-native, minimal	DynaKube operator deployment
Metric granularity	1-minute	10-60 seconds
Distributed tracing	X-Ray integration	Built-in PurePath
AI-powered root cause	Basic alarms	Dynatrace Intelligence automatic root cause
Code-level visibility	None	Full (method-level)
Log analysis	CloudWatch Logs Insights	Grail-powered DQL
Cost model	Per metric, per log GB	DPS/DDU-based
Cross-stack correlation	Limited	Automatic (host → process → service → trace)

When to Use Both

- Keep Container Insights for **EKS control plane logging** (API server, authenticator, scheduler)
- Use Dynatrace for **workload monitoring**, **distributed tracing**, and **AI-powered alerting**

- Forward Container Insights logs to Dynatrace via **CloudWatch log forwarding** for unified analysis

7. Namespace Cost Allocation

One of the key benefits of Kubernetes monitoring is the ability to allocate costs by namespace, team, or application.

CPU-Based Cost Allocation by Namespace

```
// Namespace CPU usage as percentage of total cluster CPU
timeseries nsCpu = avg(dt.kubernetes.container.cpu_usage), from:-24h, by:
{k8s.namespace.name}
| fieldsAdd avgCpu = arrayAvg(nsCpu)
| sort avgCpu desc
| limit 15
```

Memory-Based Cost Allocation by Namespace

```
// Namespace memory usage over the last 24 hours
timeseries nsMem = avg(dt.kubernetes.container.memory_working_set),
from:-24h, by:{k8s.namespace.name}
| fieldsAdd avgMemBytes = arrayAvg(nsMem)
| fieldsAdd avgMemGB = avgMemBytes / 1073741824.0
| sort avgMemGB desc
| limit 15
```

Cost Allocation Best Practices

Practice	Description
Enforce namespace labels	Require `team`, `cost-center`, `environment` labels on all namespaces
Set resource requests/limits	Accurate requests enable fair cost attribution
Use Dynatrace segments	Create segments per team/namespace for filtered dashboards

Practice	Description
Track over time	Use `makeTimeseries` to trend namespace costs weekly

8. Summary and Next Steps

Key Takeaways

- EKS monitoring requires **DynaKube Operator** for workload visibility and **cloud integration** for control plane metrics
- **Fargate** pods need ApplicationMonitoring mode with `useCSIDriver: false`
- Dynatrace provides deeper visibility than CloudWatch Container Insights, especially for **distributed tracing** and **AI root cause analysis**
- **Namespace-level metrics** enable team-based cost allocation

Next Steps

- **CLOUD-04: AWS Lambda & Serverless** — Monitoring serverless workloads
- **CLOUD-07: CloudWatch Log Ingestion** — Forwarding EKS logs to Dynatrace
- See the **K8S notebook series** for general Kubernetes monitoring patterns

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.